Coffee Quality Prediction: SVM & XGBoost Analysis

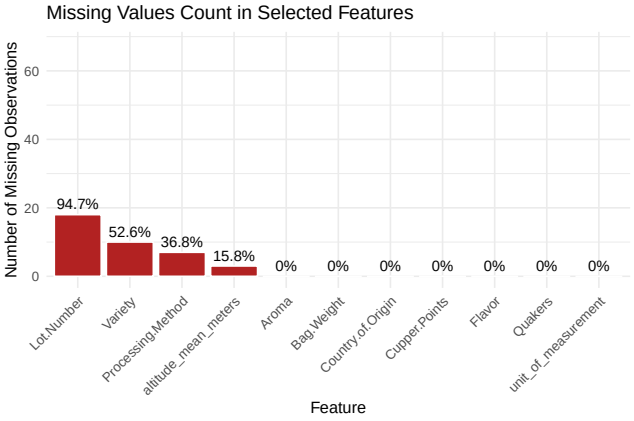
2026-01-03

1 Introduction

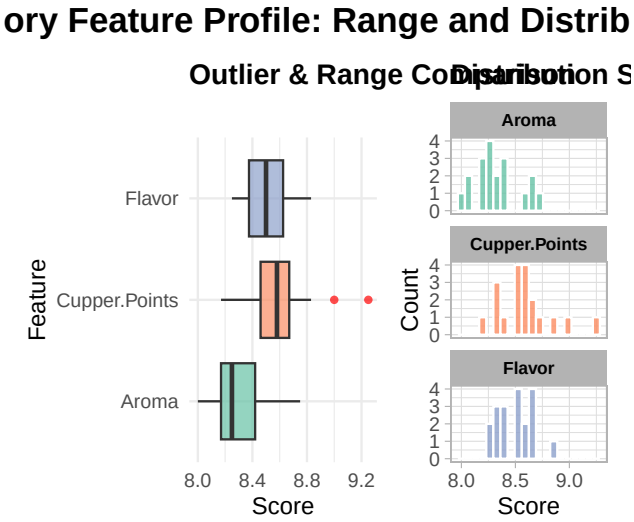
This report explores the Coffee Quality dataset, walking through everything from initial descriptive analysis to the math and application of machine learning. Our goal is to use these models to move beyond simple numbers and extract real, meaningful insights into what defines a high-quality cup of coffee.

1.1 Descriptive Analysis

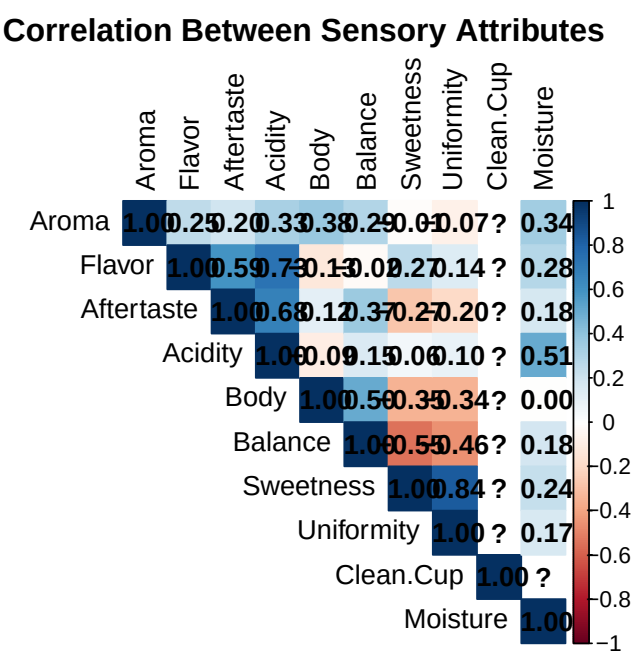
Summary of the Descriptive Analysis: The dataset has 1,311 coffee evaluations (observations) with 44 different variables. They are a mix of sensory scores (e.g. Aroma, Acidity, etc.) and environmental data (e.g. Region, Altitude). Out of these, **24 variables are categorical and 20 are numerical**. For our project, we decided to use "Cupper Points" as our target variable. "Cupper Points" is a score reflecting how much the taster enjoyed an individual coffee. We avoided "Total Cup Points" as target variable as it is just a linear combination of other sensory scores and hence not suitable for analysis.



Dealing with Missing Data One of the first things we noticed was that a lot of the categorical (text) data is missing. Because there were so many gaps, it was hard to use many of these variables in our models. Instead, we focused our energy on the variables that really mattered and tried to fill in the blanks for critical info like Altitude.



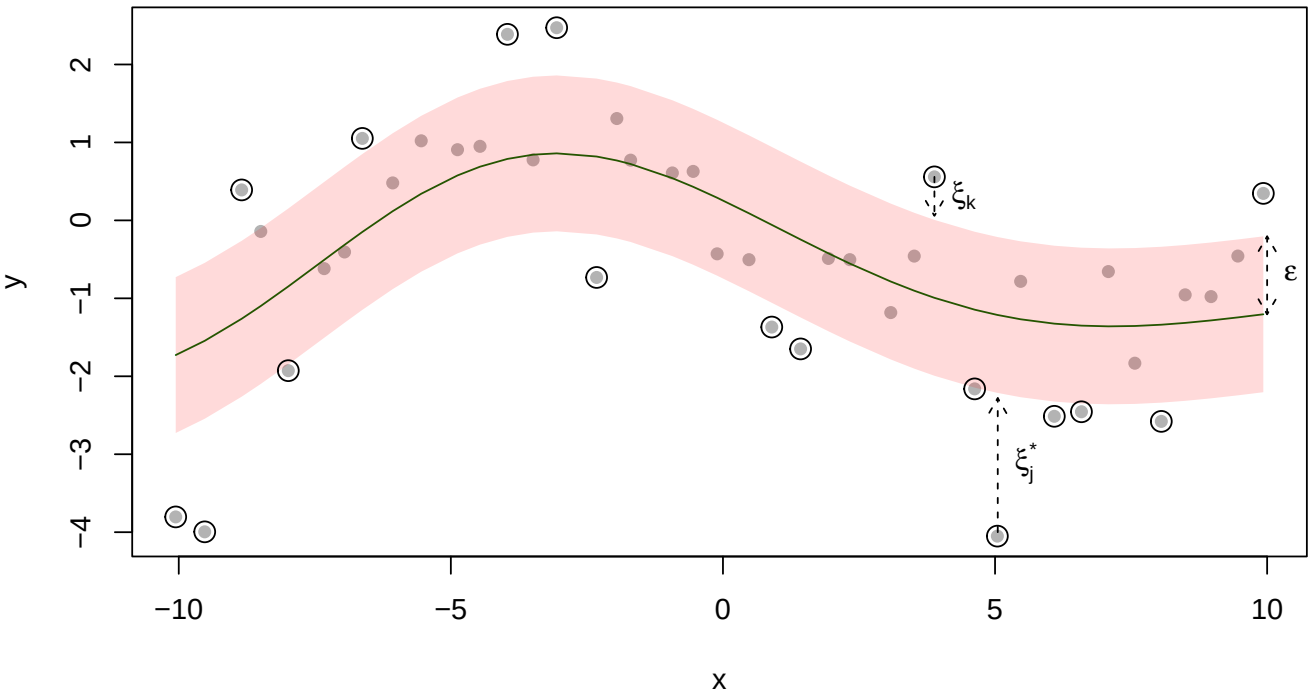
The Distribution of the Variables When we looked at the histograms and boxplots for the sensory variables, we saw that they all look very similar. Most of the scores for things like Aroma or Flavor are clustered closely together, usually between 7 and 8. We also noticed several outliers*(coffee samples that scored much higher or lower than the rest)*.



Correlation Between Numerical Features showed us how these variables lean on each other. Most sensory features are strongly linked. if a coffee smells great (Aroma), it almost always tastes great (Flavor). However, some things, like Moisture, didn't follow the crowd. Moisture has a very low correlation with the other scores and even shows a slight trend, meaning more moisture sometimes leads to a slightly lower quality score.

2 A Brief Mathematical Overview of SVM and XGBoost

2.1 Support Vector Machine for Regression



Support Vector Regression expands Support Vector Machines from discrete classification to a continuos regression model (Drucker et al. 1996).

For the regression task, instead of fitting a hyperplane, we are trying to fit a (multidimensional) function $f(x)$ to the data. Similar as in the classification task, we draw a margin around the function. However, now the margin ε is fixed at the start. (Note that ε here, is not the slack variable as the notation is commonly used for SVM in classification, e.g. in "An Introduction to Statistical Learning" (James et al. 2021a), but rather defines the width of the margin. Also, we do not aim to optimize the margin boundaries here, but instead fix them before computation starts). We want to find a function f(x) that is as flat as possible, while fitting all points into the required margin. Again, we allow for some slack

using slack variables. A wider margin decreases the risk of overfitting, while a smaller margin captures more intricacies of the data.

Mathematically speaking, want to find a function $f(x)$ with $|f(x_i)-y_i| \leq \varepsilon$ for all i. To avoid overfitting, we also introduce the constraint of making the function as flat as possible. "Flatness" in this context is a measure of how sensitive the function is to change. For a linear function $f(x) = x^\top \beta + b$ increasing this flatness can be expressed through minimizing the norm of the function gradient: $\|\beta\|$. Minimizing the expression $\frac{1}{2}\|\beta\|^2$ leads to the same optimum, while allowing for more elegant mathematical solutions, and is therefore used for computation.

Again, we want to allow for some slack, so as in the SVM for classification, we are introducing slack variables ξ_i and ξ_i^*. These slack variables allow for some room for margin violations. The optimization problem we want to solve is to minimize:

$$\frac{1}{2}\|\beta\|^2 + K\sum_{i=1}^{n}(\xi_i + \xi_i^*)$$

subject to:

$$y_i - f(x_i) \leq \varepsilon + \xi_i,$$
$$f(x_i) - y_i \leq \varepsilon + \xi_i^*$$

and

$$\xi_i > 0, \xi_i^* > 0 \quad \forall i$$

where K is fixed and β, ξ and ξ^* are variable. The factor K is introduced as cost for regularization purposes. It is choosen a priori to determine how strong the tradeoff between flatness and overfitting should be. A larger K leads to a stronger impact of the term, therefore penalizing stronger margin violations more heavily and resulting in a less flat function curve (higher bias, lower variance). Conversly, a smaller K leads to a flatter curve (lower bias, higher variance).

As in the SVM for classification, this can be generalized to non-linear functions by transforming the features using kernels.

2.2 XGBoost

XGBoost is an algorithm based on gradient boosted trees and second-order approximation (Chen 2016). It is widely used in machine learning due to its flexibility and strong empirical performance. These properties arise from a combination of algorithmic and systems-level optimizations, including efficient handling of sparse input data, approximate tree construction via weighted quantile sketches, and parallelized tree learning. In addition, careful optimization of memory access, data compression, and out-of-core computation enables the method to scale to very large datasets while maintaining computational efficiency.

This section describes the mathematical foundations of the algorithm, namely ensemble methods, regularization, and gradient boosting machines combined with second order approximation.

Tree Ensembles and Regularization

XGBoost models the prediction as an additive ensemble of K functions:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i), \quad f_k \in \mathcal{F},$$

where each f_k is a regression tree belonging to the function space $\mathcal{F}$ (Chen 2016; James et al. 2021b). Each tree acts as a weak learner that partitions the feature space into disjoint regions and assigns a constant prediction to each leaf (James et al. 2021b).

The model is trained by minimizing a regularized objective function of the form

$$\mathcal{L} = \sum_i \ell(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k),$$

where $\ell(\cdot)$ is a differentiable loss function and $\Omega(f)$ is a regularization term that penalizes model complexity (Chen 2016). This term is defined as

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2,$$

with T denoting the number of leaves in the tree and w_j the weight of leaf j. The regularization term encourages simpler trees and smooth leaf weights, helping to prevent overfitting.

The ensemble structure allows the model to reduce bias by combining multiple weak learners, while regularization controls variance by limiting tree complexity and the magnitude of leaf weights (James et al. 2021b). This balance between bias and variance enables XGBoost to achieve strong predictive performance while maintaining good generalization.

Gradient Boosting Machines

The tree ensemble is constructed in a stage-wise manner, where weak learners are added iteratively to improve the model (Friedman 2001). At iteration t, the prediction is given by

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i),$$

with $f_t \in \mathcal{F}$ denoting the newly added regression tree.

Gradient tree boosting can be interpreted as gradient descent in function space, where each new learner is chosen to minimize the loss function in the direction of the negative gradient (Friedman 2001). Specifically, the new tree at iteration t is trained to fit the negative first order gradient

$$-g_i^{(t)} = -\left.\frac{\partial \ell(y_i, \hat{y})}{\partial \hat{y}}\right|_{\hat{y}=\hat{y}_i^{(t-1)}}.$$

XGBoost extends this framework by explicitly approximating the objective function using a second-order Taylor expansion of the loss around the current predictions:

$$\ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) \approx \ell(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2,$$

where

$$h_i = \left.\frac{\partial^2 \ell(y_i, \hat{y})}{\partial \hat{y}^2}\right|_{\hat{y}=\hat{y}_i^{(t-1)}}$$

is the second order derivative of the loss (Chen 2016). By incorporating both first- and second-order information, XGBoost enables efficient optimization of tree structures and leaf weights within each boosting iteration.

3 Support Vector Regression

3.1 Data Preprocessing and Cleaning

3.1.1 Feature selection

In SVM Regression, categorical variables are One-Hot Encoded using dummy variables. This means, features with many unique values will lead to a sparse feature matrix. This is disadvantageous as it increases computational cost and poses the risk of overfitting. Additionally, categories with only a few or even only one observation most likely will not add to model robustness, but are adding noise to the signal. Therefore, we are removing categorical features with many unique values, only keeping columns with a low number of unique values. E.g. we are not including the "Region" column with 344 unique values in the model training, because we'd on average have less than 4 observations per Region. For features with a high number of missing values, we either remove the column or impute the missing

values. For instance for the "Variety" column, we have about 15% missing values and 8% entries "other" at 30 different varieties. Keeping those, would mean adding 30 sparse columns, and at the same time the information gain is most likely not too high with this many missing values. Therefore, we are not including this feature.

3.1.2 Imputation

For the altitude column, we have about 17% missing values. Here, we are imputing the missing values from the "Region" column. For each observation with missing altitude, we are imputing the median altitude of all coffees from the same region. This covers almost all missing values. For the remaining observations, we are imputing the mean altitude of all coffees from the same country of origin. As the "Region" and "Country" columns are not used in the model training, we are not creating correlated features with this imputation. We are imputing the missing values after the train/test split, using only values from the training set, to avoid data leakage from the test set into the training set.

3.1.3 Further preprocessing

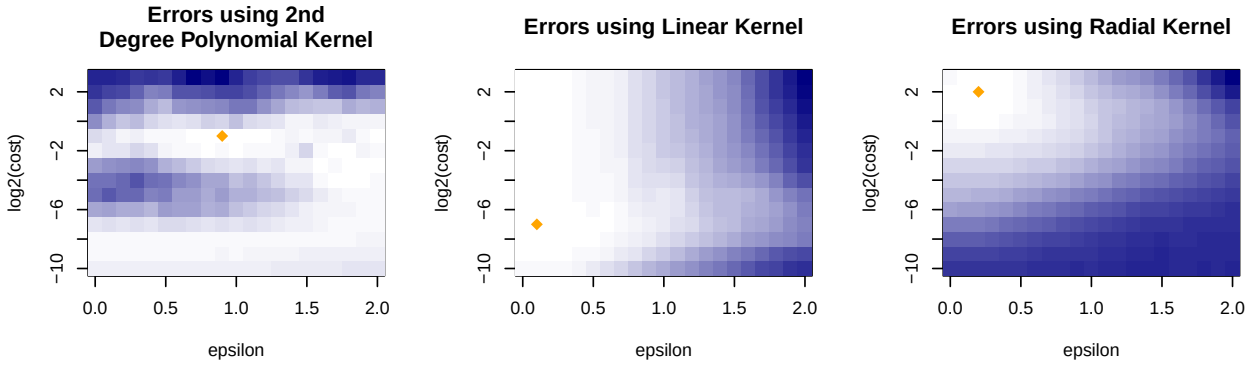
The bag weight column contains both pounds and kilograms, so we are converting all to kilograms. However, for some entries, it is unclear which unit is used. For these, we are averaging the weight in pounds and kilograms. This means for those entries the weight is certainly incorrect, but should be within an order of magnitude of the correct weight. If we can assume that speciality coffee is sold in smaller quantities, we can still gain valuable information from the weight.

3.1.4 Scaling

The SVM method from the e1071 packaged used for model training has a built in scaling functionality. Both predictor and target variables are scaled to have expected value 0 and variance 1. (Meyer et al. 2025) This transformation is based on the training data and applied to train and test data sets.

3.2 Hyperparameter Optimization

For the prediction of cupper points we train an SVM for regression on the data set, holding out the test data set for final evaluation. To choose appropriate hyperparameters, we conduct a hyperparameter optimization on the training data set using 10-fold cross-validation. We use a grid search for different values for cost and epsilon for a radial, linear, and 2-degree polynomial kernel. We compare the mean absolute error between runs to choose the best-performing hyperparameter combination. Finally, we train a model with these hyperparameters on the entire training set and evaluate on the test data set.



These figures show the mean absolute error for the different hyperparameters by kernel. The best performing hyperparameter combination for each kernel are marked with orange points.

epsilon	cost	kernel	CV MAE
0.1	0.0078125	linear	0.208

The best performing kernel is the linear kernel, which means the relationship between the features and the target variable is best approximated by a multidimensional hyperplane.

4 XGBoost Model

4.1 Data Preprocessing

To ensure that the SVM and XGBoost models are comparable, we apply the same data preprocessing described above to XGBoost, except we skip feature scaling.

SVM requires scaling because it calculates distances between data points to find the optimal separating hyperplane. Features with larger ranges would dominate the distance calculations, which would make the model incorrect. XGBoost is a tree-based method, it is only important if values are higher or lower (e.g., "is Flavor > 7.5?"), so scaling is unnecessary.

4.2 Configuration & Data

Config	**Value**
Seed	123
Split	80/20
CV Folds	10
Early Stop	50
Max Rounds	2000

Data	**Value**
Train n	1047
Test n	262
Features	21
Target $\bar{x}$	7.5

Model	**MAE**	**R^2**
Mean	0.309	0.00
Linear	0.146	0.59
XGBoost	0.141	0.68

Baseline models

We use two baseline models: the Mean Predictor and Linear Regression.

The **Mean Predictor** has the $R^2 = 0$, which confirms that our target has meaningful variance to explain. The MAE of 0.309 means that without any features, we'd be off by about 0.3 points on average.

Linear Regression provides an interpretable baseline with R^2 of 0.59, showing that 59% of the variance in cupping scores can be explained by linear combinations of features. Any improvement from the XGBoost models means capturing non-linear relationships that linear regression misses.

4.3 Hyperparameter Optimization

eta	max_depth	subsample	colsample	rounds	CV MAE
0.05	5	0.8	0.7	127	0.149

Hyperparameter selection

We tune four main parameters to balance bias and variance:

eta (learning rate): 0.01–0.1. Smaller values make each tree contribute less, reducing overfitting but needing more rounds.

max_depth: 3–7. Controls tree complexity. Shallow trees capture simple patterns; deeper trees can model complex patterns but may overfit, especially on our dataset with about 1000 samples.

subsample: 0.7–0.9. Fraction of rows used per tree. Reduces overfitting by adding randomness. We avoid below 0.7, as it would leave too few samples per tree.

colsample_bytree: 0.7–0.9. Fraction of features per tree. Prevents any one feature from dominating.

These ranges are conservative. With about 1000 samples, extreme values (e.g., eta=0.3, max_depth=10) would overfit, but mild regularization is enough.

Lambda and Min_Child_Weight Regularization
We tested additional hyperparameters, but they were ineffective. XGBoost's lambda (L2) and min_child_weight provided no benefit because implicit regularization from learning rate, max_depth, subsample, and early stopping already prevents overfitting. Extra regularization slightly worsened the metrics, so it was omitted.

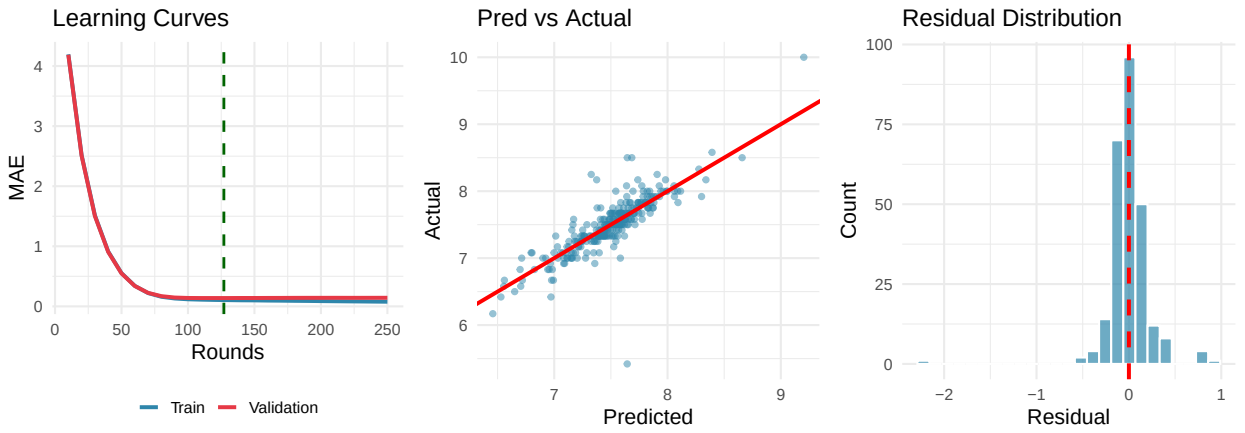
Bayesian Optimization vs Grid Search
We also tested Bayesian Optimization, but it was ineffective. With only 4 parameters and a small search space, it added overhead, slowed hyperparameter tuning, and offered no meaningful improvement. Grid search is sufficient for this dataset and model.

10-fold CV: Matches SVM for fair comparison. Each fold has about 100 samples, giving stable MAE estimates. Fewer folds increase variance; leave-one-out is too slow.

Early stopping (50 rounds): Stops training if validation MAE doesn't improve for 50 rounds, automatically choosing the optimal number of trees and avoiding overfitting.

```
## XGBoost model trained and saved to: saved_models/model_xgb_comparable.rds
```

4.4 Diagnostic Plots



The **learning curves** show that the model converges with minimal overfitting. In the first 50 rounds, training and validation MAE drop as the model captures the main signal. After that, improvement slows and additional rounds provide little benefit, which justifies early stopping. The small train-validation gap of 0.021 MAE indicates that overfitting is minimal.

The **predictions vs actual** plot shows that most points lie close to the diagonal, indicating accurate predictions. Larger errors appear at the highest and lowest quality scores, which is expected because extreme values are rare and the model has fewer examples to learn from. There is no visible systematic bias, as points are evenly distributed around the diagonal.

The **residuals** are centered around zero, confirming that the prediction bias is minimal. Their roughly bell-shaped distribution suggests that errors are mostly random rather than systematic, meaning the model has captured the main signal.

4.5 Conclusions

Model performance: XGBoost achieves an MAE of 0.141, meaning predictions are usually within 0.14 points of the true cupper score on a 10-point scale. This is a small but consistent improvement over linear regression (MAE 0.146). The R^2 increases from 0.59 to 0.68, showing that XGBoost explains about 9% more variance. This suggests most relationships are linear, with XGBoost mainly capturing minor non-linear effects and interactions.

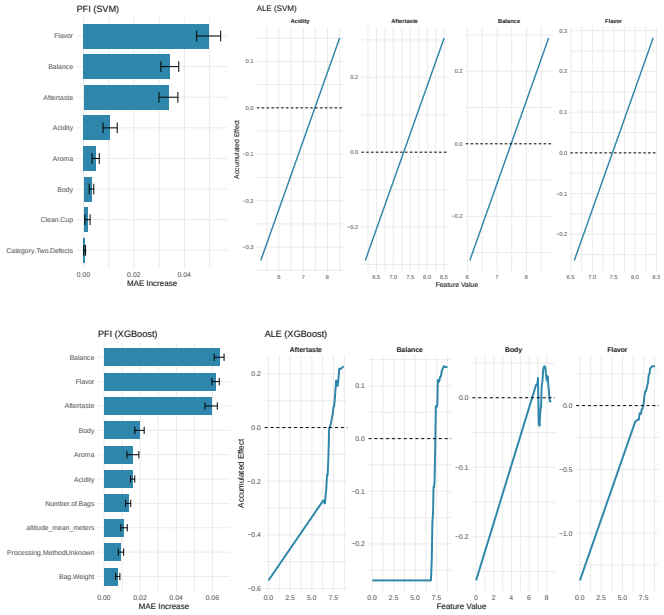
Limitations: Performance is weaker at extreme quality scores due to limited data. The dataset size (about 1300 samples) may restrict learning of complex patterns, and temporal effects such as harvest

year are not modeled (because of the issues with source data). Finally, the target variable cupper points is subjective and includes human rating variability.

4.6 Explainable AI

4.6.1 Permutation Feature Importance (PFI)

PFI measures how much model performance degrades when a feature's signal is destroyed by shuffling (Friedman 2001). Unlike XGBoost's native Gain metric, PFI is model-agnostic and accounts for feature interactions.



The top features are all sensory scores, confirming the model relies on expert taste evaluation. PFI provides a model-agnostic view that complements XGBoost's native importance (gain). Both agree that Flavor is most predictive.

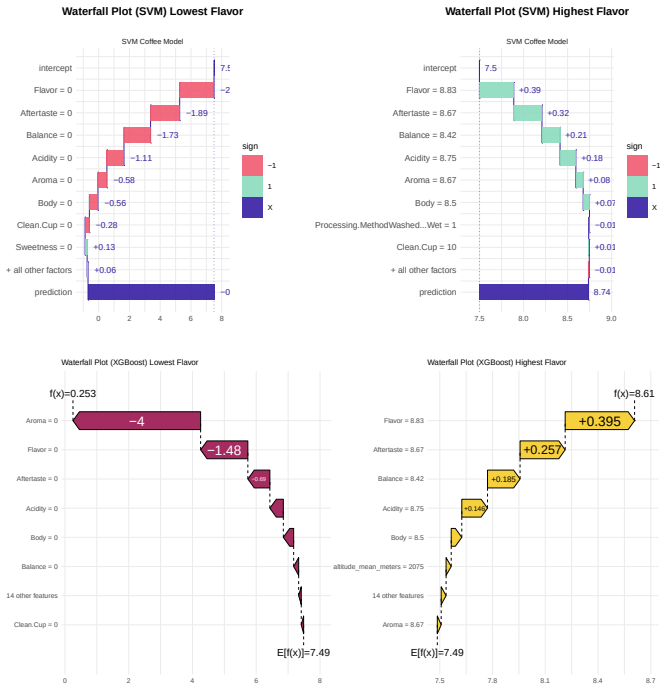
4.6.2 Accumulated Local Effects (ALE)

ALE plots show the marginal effect of a feature on the model's predictions by averaging local changes in the prediction over the distribution of the other features (Apley and Zhu 2020). It is chosen here instead of PDP (Partial Dependence plots), because it is more robust to multicollinearities and centered around 0. We plot the top four features ranked by permutation feature importance (PFI).

All sensory features show the expected pattern: higher scores lead to higher predicted quality. The PDP range for Flavor is largest, indicating it has the strongest marginal effect on predictions.

4.6.3 Shapley Additive Explanations (SHAP)

SHAP values provide a unified measure of feature importance based on cooperative game theory (Lundberg and Lee 2017). For each prediction, SHAP assigns each feature a value that represents its contribution to moving the prediction away from the baseline (average prediction). We utilize the SHAP values to inspect the microscopic effects of the feature values for two specific observations. For this we chose one observation with a high value for Flavor and one with a low value, since this is the most important feature according to PFI.

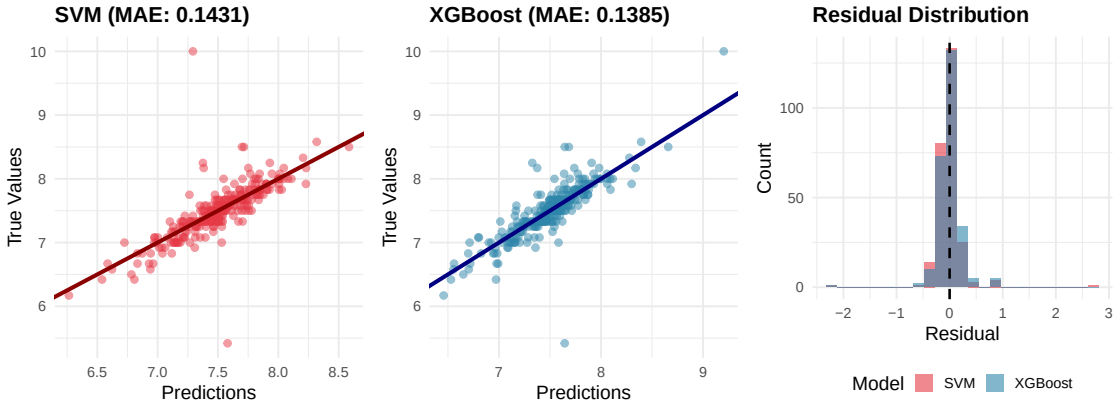


XGBoost acts like a coffee expert because it treats zero scores for Flavor or Aroma as "deal-breakers". In its waterfall plot, the prediction drops sharply from 7.5 to 1.83. It understands that if the core features are missing, the quality score should collapse, not just decrease slightly.

The SVM model is much more cautious and stays close to the average. Even with zero sensory scores, it only drops the final prediction to 7.33. Because it is a Linear SVM, it simply subtracts a small, fixed amount for each feature. Since most coffees in our data are high-quality, the model hasn't learned to penalize zero scores heavily enough and stays near its 7.48 starting point.

5 Model Comparison - SVM vs XGBoost

Model	MAE	RMSE	R2
SVM (Linear Kernel)	0.1431	0.2790	0.5855
XGBoost	0.1385	0.2368	0.7015



Both models achieve similar predictive accuracy, with XGBoost having slightly smaller MAE (0.141 vs 0.146) and larger R^2 (0.68 vs 0.59). The scatter plots confirm that both models produce predictions closely aligned with true values, with no systematic bias evident in either case. The residual distributions are nearly identical and centered around zero, indicating that both approaches effectively capture the underlying signal in the data.

5.1 Literature

Apley, Daniel W, and Jingyu Zhu. 2020. "Visualizing the Effects of Predictor Variables in Black Box Supervised Learning Models." *Journal of the Royal Statistical Society Series B: Statistical Methodology* 82 (4): 1059–86.

Chen, Tianqi. 2016. "XGBoost: A Scalable Tree Boosting System." *Cornell University*.

Drucker, Harris, Christopher J. C. Burges, Linda Kaufman, Alex Smola, and Vladimir Vapnik. 1996. "Support Vector Regression Machines." In *Advances in Neural Information Processing Systems*, edited by M. C. Mozer, M. Jordan, and T. Petsche. Vol. 9. MIT Press. https://proceedings.neurips.cc/paper_files/paper/1996/file/d38901788c533e8286cb6400b40b386d-Paper.pdf.

Friedman, Jerome H. 2001. "Greedy Function Approximation: A Gradient Boosting Machine." *Annals of Statistics*, 1189–1232.

James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021b. *An Introduction to Statistical Learning: With Applications in r*. 2nd ed. Springer. https://doi.org/10.1007/978-1-0716-1418-1.

———. 2021a. *An Introduction to Statistical Learning: With Applications in r*. 2nd ed. New York: Springer.

Lundberg, Scott M, and Su-In Lee. 2017. "A Unified Approach to Interpreting Model Predictions." *Advances in Neural Information Processing Systems* 30.

Meyer, David, Evgenia Dimitriadou, Kurt Hornik, Andreas Weingessel, and Friedrich Leisch. 2025. *E1071: Misc Functions of the Department of Statistics, Probability Theory Group (Formerly: E1071), TU Wien*. https://CRAN.R-project.org/package=e1071.